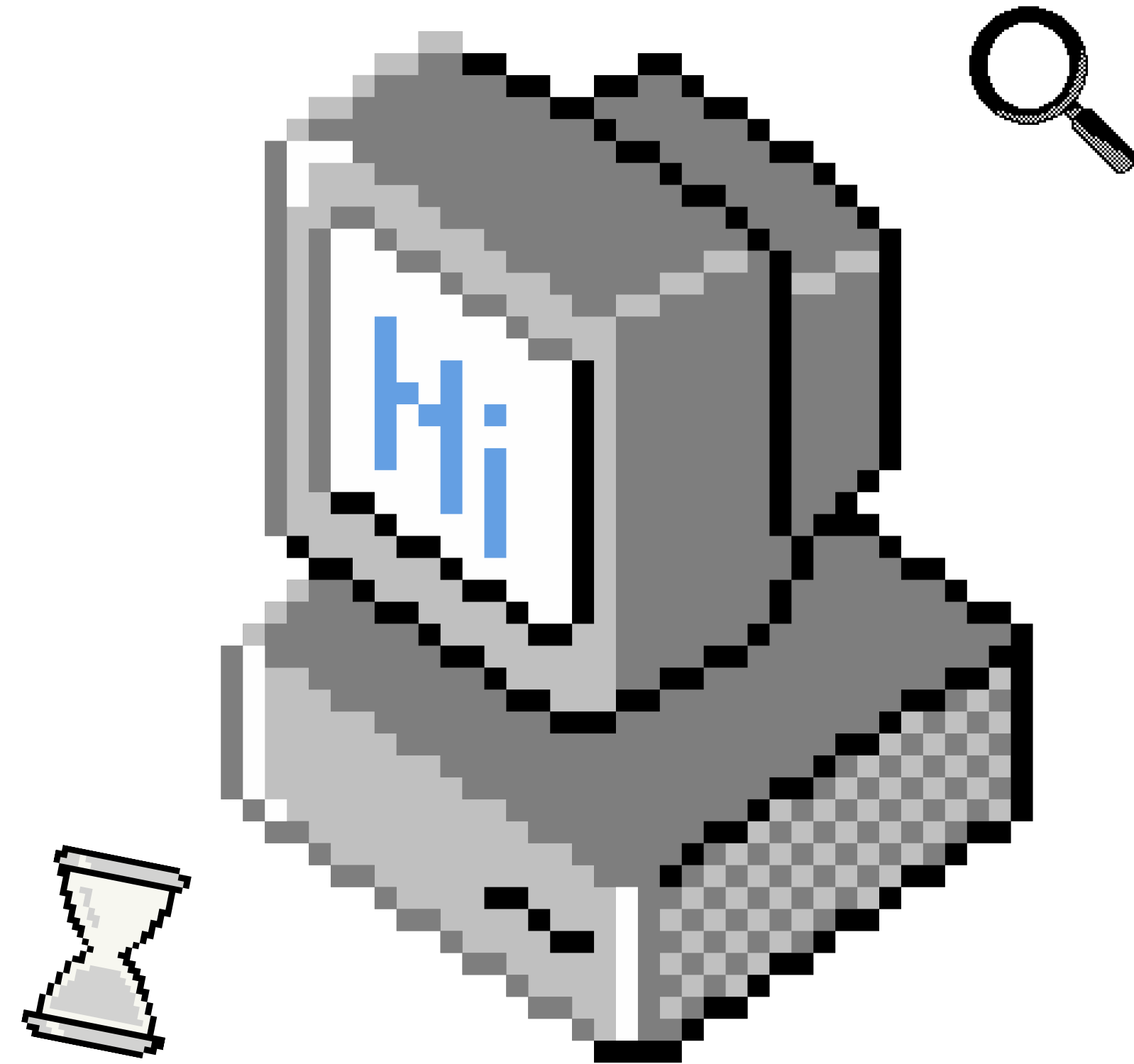




PRÁCTICA 5

Equipo 6

ACTIVIDAD 1

Es conveniente usar métodos modificadores en los constructores?

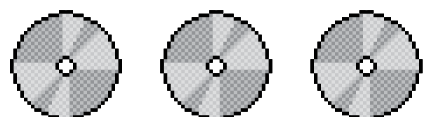
Por lo general, no es una buena práctica usar métodos modificadores (setters) dentro de los constructores. ¿Por qué? El objetivo principal de un constructor es darle un estado inicial válido a un objeto recién creado. Si llamamos a un método 'set' desde el constructor y, en el futuro, otra clase hereda de la nuestra y modifica (sobrescribe) ese 'set', podríamos causar errores graves. Esto ocurre porque el método modificado se ejecutaría antes de que el objeto termine de construirse por completo. Por lo tanto, lo más seguro y recomendado es asignar los valores directamente a las variables (usando `this.variable = valor;`) dentro del constructor, garantizando así que el objeto nazca con los datos correctos sin depender de métodos externos."

ACTIVIDAD 2

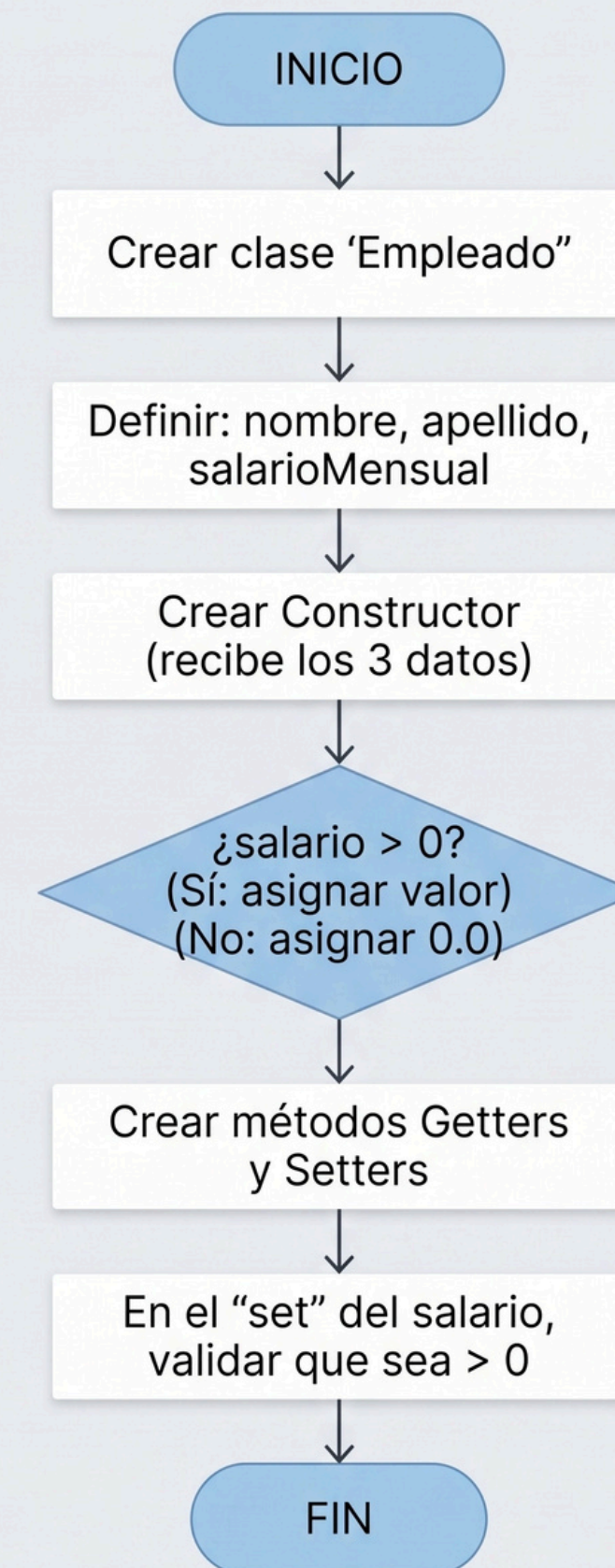
El objetivo aquí es crear un empleado con nombre, apellido y salario, asegurándonos de que el salario no sea un número negativo.

Algoritmo

1. Crear la clase Empleado.
2. Declarar los atributos privados: nombre (texto), apellido (texto) y salarioMensual (decimal).
3. Crear el constructor que reciba estos 3 datos. Aquí agregamos una regla (un if): si el salario es mayor a 0, se lo asignamos; si no, lo dejamos en 0.
4. Crear los métodos get (para leer) y set (para modificar) de cada variable. En el set del salario, volvemos a poner la regla de que debe ser positivo.



Algoritmo: Clase Empleado



Ejercicio 2 - Clase Empleado

Clase Empleado

Atributos privados:

nombre: Cadena

apellido: Cadena

salarioMensual: Real

Método Constructor Empleado(param_nombre: Cadena,
param_apellido: Cadena, param_salarioMensual: Real)
setNombre(param_nombre)
setApellido(param_apellido)
setSalarioMensual(param_salarioMensual)

Fin Método

Método setNombre(param_nombre: Cadena)
nombre <- param_nombre

Fin Método

Método setApellido(param_apellido: Cadena)
apellido <- param_apellido

Fin Método

Método setSalarioMensual(param_salarioMensual: Real)
Si param_salarioMensual > 0 Entonces
salarioMensual <- param_salarioMensual

Fin Si

Fin Método

Método getNombre() Retorna Cadena
Retornar nombre

Fin Método

Método getApellido() Retorna Cadena
Retornar apellido

Fin Método

Método getSalarioMensual() Retorna Real
Retornar salarioMensual

Fin Método

Fin Clase

PSEUDOCÓDIGO (ACT. 2)

Ejercicio 2 - Clase MainEmpleado

Clase MainEmpleado

Método Principal()

e1: Empleado <- Nuevo Empleado("Ana", "Lopez", 12000)
e2: Empleado <- Nuevo Empleado("Luis", "Perez", 15000)

// Salario anual

Mostrar "Salario anual e1: ", e1.getSalarioMensual() * 12

Mostrar "Salario anual e2: ", e2.getSalarioMensual() * 12

// Aumento del 10%

e1.setSalarioMensual(e1.getSalarioMensual() * 1.10)

e2.setSalarioMensual(e2.getSalarioMensual() * 1.10)

// Nuevo salario anual

Mostrar "Nuevo salario anual e1: ", e1.getSalarioMensual() * 12

Mostrar "Nuevo salario anual e2: ", e2.getSalarioMensual() * 12

Fin Método

Fin Clase

ACTIVIDAD 3

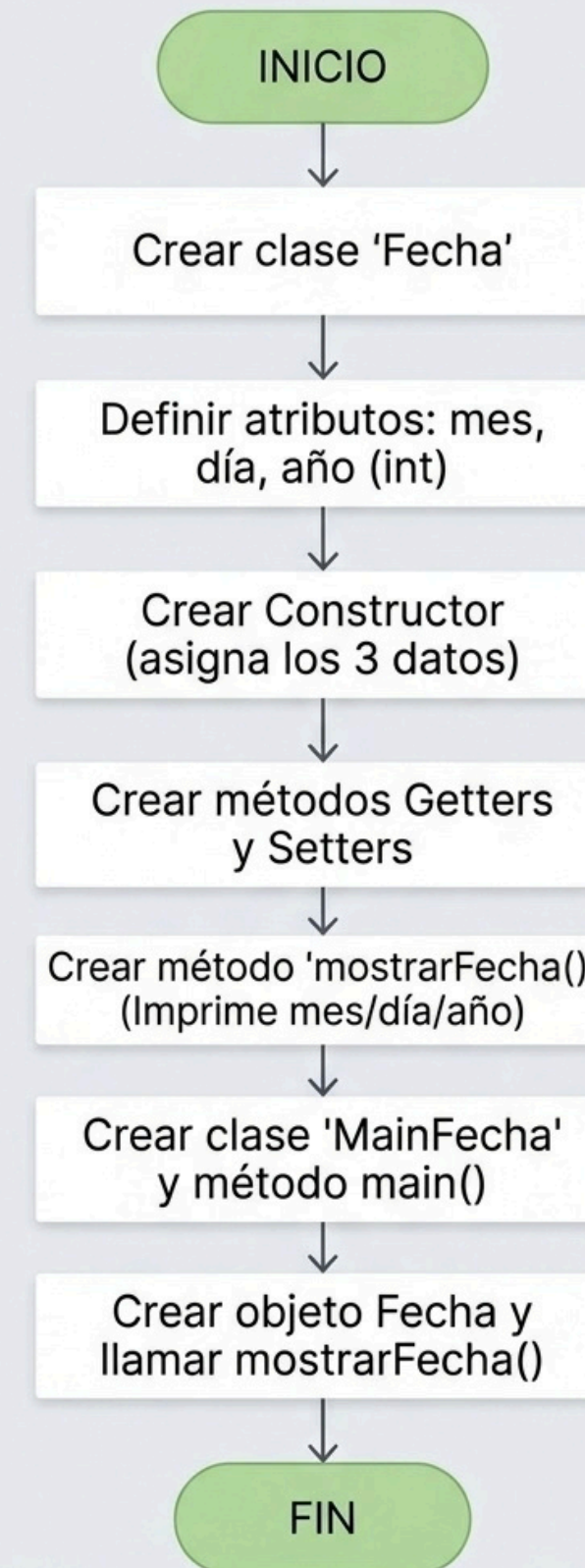
Clase Fecha y MainFecha

Esta actividad pide crear una clase para manejar fechas y otra clase principal para probarla, asumiendo que los datos que el usuario meta siempre serán correctos.

Algoritmo :

1. Crear la clase Fecha con los atributos numéricos (enteros): mes, día y año.
2. Crear un constructor que reciba estos 3 datos y los asigne directamente a los atributos.
3. Crear los métodos get y set para cada uno de los atributos.
4. Crear un método llamado mostrarFecha que simplemente imprima en pantalla los números separados por una diagonal /.
5. Crear una segunda clase llamada MainFecha con el método principal (main) para crear una fecha de prueba y mostrarla en pantalla.

Algoritmo: Clase Fecha



Ejercicio 3 - Clase Fecha

Clase Fecha

Atributos privados:

mes: Entero
dia: Entero
anio: Entero

// Se asume que los valores son correctos

Método Constructor Fecha(param_mes: Entero,
param_dia: Entero, param_anio: Entero)
setMes(param_mes)
setDia(param_dia)
setAnio(param_anio)

Fin Método

Método setMes(param_mes: Entero)

mes <- param_mes

Fin Método

Método setDia(param_dia: Entero)

dia <- param_dia

Fin Método

Método setAnio(param_anio: Entero)

anio <- param_anio

Fin Método

Método getMes() Retorna Entero

Retornar mes

Fin Método

Método getDia() Retorna Entero

Retornar dia

Fin Método

Método getAnio() Retorna Entero

Retornar anio

Fin Método

Método mostrarFecha()

Mostrar mes + "/" + dia + "/" + anio

Fin Método

Fin Clase

PSEUDOCÓDIGO (ACT. 3)

Ejercicio 3 - Clase MainFecha

Clase MainFecha

Método Principal()

f: Fecha <- Nueva Fecha(9, 29, 2026)

f.mostrarFecha() // 9/29/2026

f.setMes(12)

f.setDia(25)

f.setAnio(2027)

f.mostrarFecha() // 12/25/2027

Fin Método

Fin Clase